

第二次实验：运维实验

一、实验背景

运维是确保软件系统稳定、可靠运行的关键环节。随着现代软件系统的复杂度不断增加，运维不仅仅是对系统硬件和软件的维护，更加注重实时监控、性能优化以及故障排查。监控系统的搭建和使用能够帮助运维人员及时发现系统中的潜在问题，保证系统的高可用性和优质性能。

Prometheus和Grafana是当前广泛应用于系统监控和可视化领域的两款工具。Prometheus是一种开源的监控和报警系统，能够高效地收集、存储和查询各种指标数据，特别适用于大规模分布式系统的监控。它通过拉取式数据采集模型，定期从配置的目标系统中获取监控数据，并支持丰富的查询语言（PromQL）来进行数据分析和告警。Grafana则是一个开源的数据可视化平台，可以与Prometheus无缝集成，提供实时数据的图表展示，帮助运维人员直观地了解系统的运行状态和性能指标。

为验证微服务系统在异常场景下的鲁棒性与容错能力，需要主动模拟真实故障以检验监控有效性与系统自愈能力。ChaosMesh是面向Kubernetes的混沌工程工具，支持Pod销毁、网络延迟、服务崩溃、资源压力等多种故障注入能力，可在可控环境下制造故障，配合监控系统观测故障发生、传播与恢复的全流程指标变化，为故障定位、容灾演练与稳定性优化提供真实数据支撑。

本次实验以SockShop微服务为载体，通过部署Prometheus+Grafana搭建完整监控体系，并使用ChaosMesh主动注入故障，实现监控采集 — 可视化展示 — 故障模拟 — 指标分析的全流程实践，掌握云原生环境下微服务运维、监控与稳定性测试的核心方法。

二、实验内容

下面，以SockShop为例，进行Prometheus+Grafana监控系统的部署以及使用ChaosMesh向k8s集群中注入故障。

1. Prometheus+Grafana部署

部署监控系统，方便进行系统故障的定位检测，修复。

输入以下指令，注入监控资源

代码块

```
1 kubectl create -f ./deploy/kubernetes/manifests-monitoring
```

```
D:\MinikubeProject\SockShop\microservices-demo>kubectl create -f ./deploy/kubernetes/manifests-monitoring
namespace/monitoring created
serviceaccount/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
configmap/prometheus-configmap created
configmap/prometheus-alertrules created
Warning: spec.template.spec.nodeSelector[beta.kubernetes.io/os]: deprecated since v1.14; use "kubernetes.io/os" instead
deployment.apps/prometheus-deployment created
service/prometheus created
daemonset.apps/node-directory-size-metrics created
serviceaccount/kube-state-metrics created
clusterrole.rbac.authorization.k8s.io/kube-state-metrics created
clusterrolebinding.rbac.authorization.k8s.io/kube-state-metrics created
deployment.apps/kube-state-metrics created
service/kube-state-metrics created
configmap/grafana-import-dashboards created
deployment.apps/grafana-core created
service/grafana created
job.batch/grafana-import-dashboards created
serviceaccount/node-exporter created
daemonset.apps/node-exporter created
service/node-exporter created
```

输入以下指令，查看监控服务

代码块

```
1 kubectl get svc -n monitoring
```

```
D:\MinikubeProject\SockShop\microservices-demo>kubectl get svc -n monitoring
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)              AGE
grafana              NodePort      10.98.63.34     <none>            80:31300/TCP         35s
kube-state-metrics   ClusterIP     None            <none>            8080/TCP,8081/TCP    35s
node-exporter        ClusterIP     None            <none>            9100/TCP              35s
prometheus           NodePort      10.107.213.155 <none>            9090:31090/TCP       36s
```

可以发现grafana和prometheus都是NodePort类型，也就是外部可直接访问。

输入下面两个命令，分别得到他们的IP地址

代码块

```
1 minikube service prometheus -n monitoring --url
2 minikube service grafana -n monitoring --url
```

然后进入浏览器后可以得到如下所示，这两张图分别是Prometheus和Grafana的界面：

127.0.0.1:52137/graph?g0.expr=apiserver_selfrequest_total&g0.tab=1&g0.stacked=0&g0.range_input=1h&g0.end_input=2025-02-18%2002...

Prometheus Alerts Graph Status Help Classic UI

☒ Use local time ☐ Enable query history ☒ Enable autocomplete ☐ Use experimental editor ☒ Enable highlighting ☒ Enable linter

Q apiserver_selfrequest_total Execute

Table Graph

2025-02-18 10:30:40

apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="apiservices", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="clusterrolebindings", verb="GET")	47
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="clusterrolebindings", verb="LIST")	2
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="clusterroles", verb="GET")	67
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="clusterroles", verb="LIST")	2
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="configmaps", verb="LIST")	2
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="csidrivers", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="customresourcedefinitions", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="endpoints", verb="GET")	9
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="endpoints", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="endpoints", verb="PUT")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="endpointslices", verb="GET")	9
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="endpointslices", verb="PUT")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="flowschemas", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="ingressclasses", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="leases", verb="GET")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="leases", verb="LIST")	1
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="leases", verb="PUT")	9
apiserver_selfrequest_total(instance="192.168.58.2:8443", job="kubernetes-apisservers", resource="limitranges", verb="LIST")	2

Prometheus Alerts Graph Status Help Classic UI

Targets

All Unhealthy Collapse All

kube-state-metrics (1/1 up)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://kube-state-metrics.monitoring.svc.cluster.local:8080/metrics	UP	instance="kube-state-metrics.monitoring.svc.cluster.local:8080" job="kube-state-metrics"	33.785s ago	9.110ms	

kubernetes-apisservers (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
https://192.168.49.2:8443/metrics	UP	instance="192.168.49.2:8443" job="kubernetes-apisservers"	34.131s ago	253.466ms	

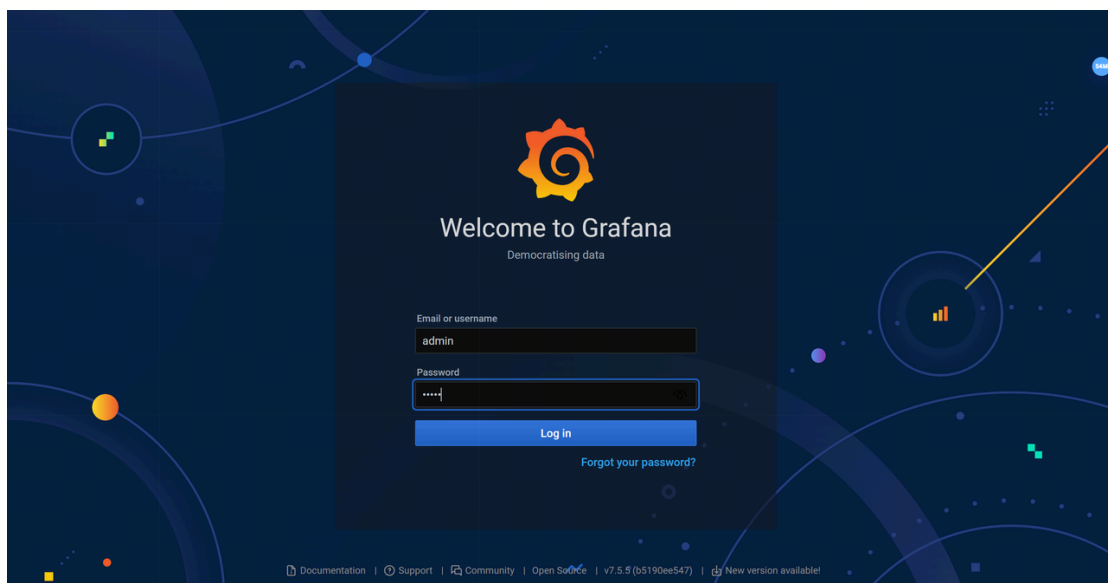
kubernetes-cadvisor (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
https://kubernetes.default.svc/api/v1/nodes/minikube/proxy/metrics/cadvisor	UP	beta.kubernetes.io.arch="amd64" beta.kubernetes.io.os="linux" instance="minikube" job="kubernetes-cadvisor" kubernetes.io.arch="amd64" kubernetes.io.hostname="minikube" kubernetes.io.os="linux" minikube.k8s.io.commit="d45d320e41b5451cdf3c01891bc4e13d189586ed-dirty"	34.137s ago	212.364ms	

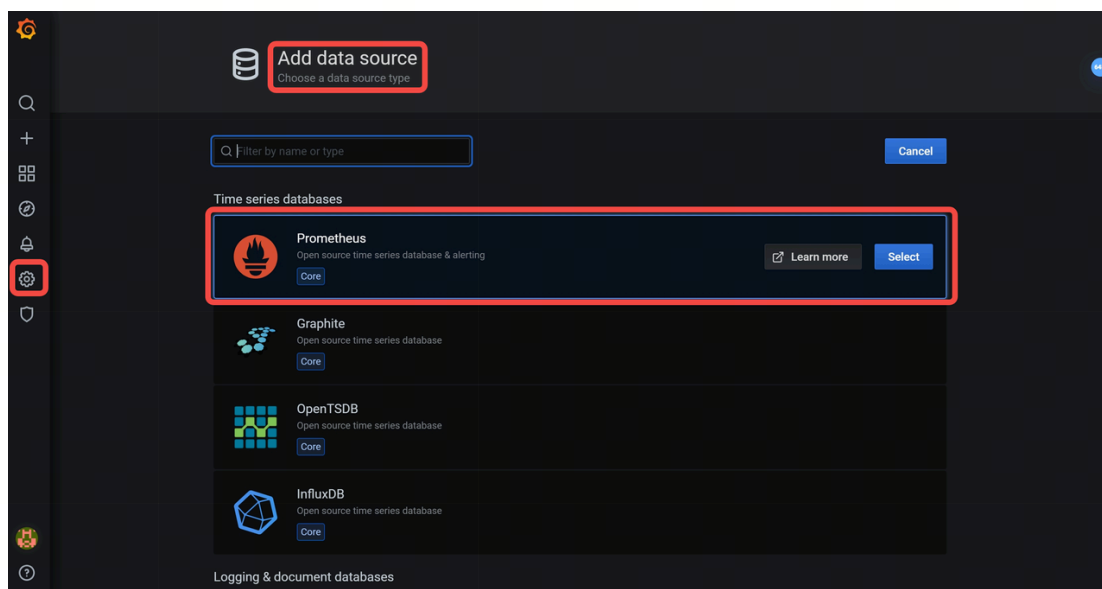
127.0.0.1:56932/targets

Grafana:

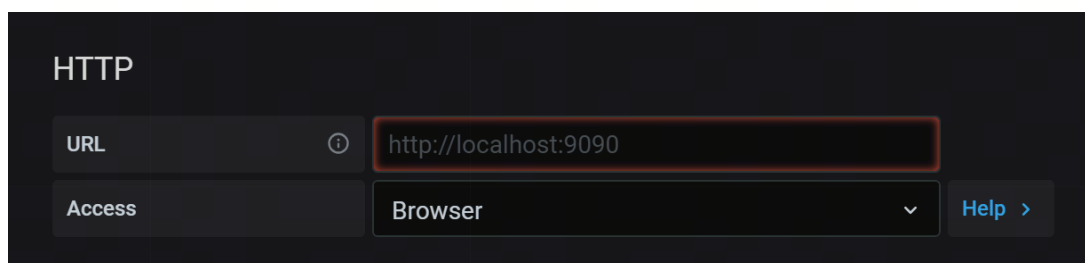
Grafana的初始默认用户名和密码都是admin



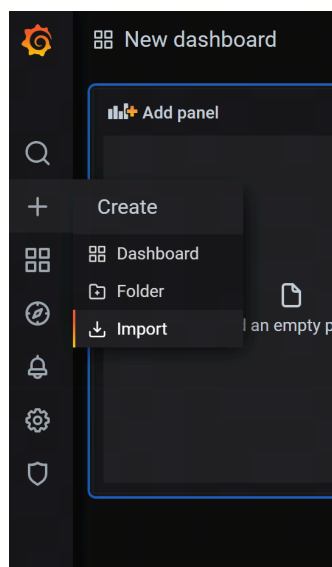
登录后在设置中添加数据来源：



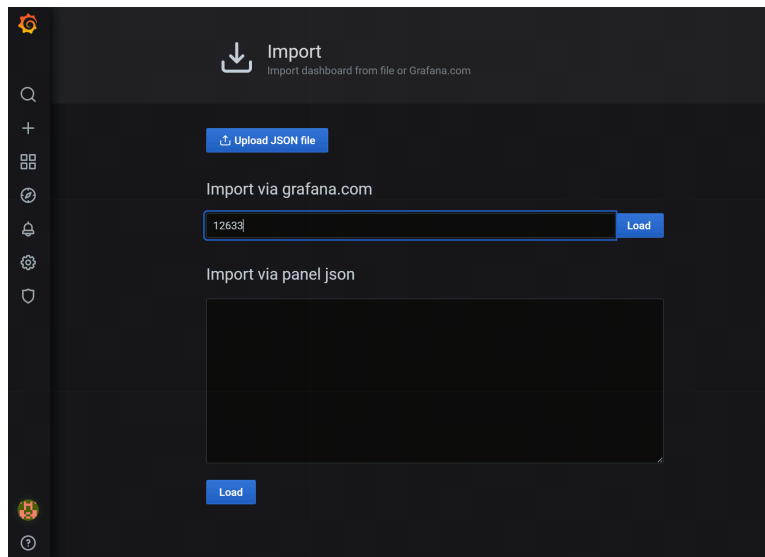
添加 Prometheus 的 URL，并点击底部的 Save&Test



添加处选择导入 Import



此处 ID 号可以通过去 grafana.com 查询自己想要可视化样式的 ID



还可以查询仪表盘ID生成仪表盘样式：



1.1 探索

关于Prometheus+Grafana的其他使用请同学们自行探索~

下面贴出相关资料，方便大家了解Prometheus+Grafana：

https://blog.csdn.net/qq_31725371/article/details/114697770

2. ChaosMesh注入故障

PS. 需要使用ChaosMesh注入故障的直接原因是因为正常部署的微服务系统可能没有故障，在后续使用复现的算法进行故障检测或定位时可能无法检测和定位到故障。

在部署完微服务系统和监控系统后，需要学习部署ChaosMesh以注入故障。

ChaosMesh，它是一个用于 **Kubernetes 环境** 的混沌工程工具。你们可以通过它向自己部署的**微服务系统**注入故障，比如模拟服务崩溃、网络延迟或存储问题。

这样做的目的是 **测试系统的鲁棒性**，也就是验证系统在遇到故障时是否能自我恢复或者保持稳定。通过这种方式，我们可以监控故障后的系统状态，并且 **导出监控数据进行检测**，帮助我们更好地发现系统潜在的问题。

简单来说，**ChaosMesh** 让你故意制造一些“破坏”，目的是确保在真实故障发生时，系统能够恢复或继续正常运行。

希望大家通过使用 ChaosMesh，能更好地理解微服务系统的容错性和如何提高系统的稳定性！

更详细的部署文档信息请参考：

[Chaos Mesh 简介](#) | [Chaos Mesh](#)

1. 安装Helm

如想系统性了解，请参阅官方安装文档：<https://helm.sh/zh/docs/intro/install/>

便捷方法：

这里提供两个下载好的压缩包：



helm-v4.1.4-windows-amd64.zip

18.90MB



helm-v4.1.4-windows-arm64.zip

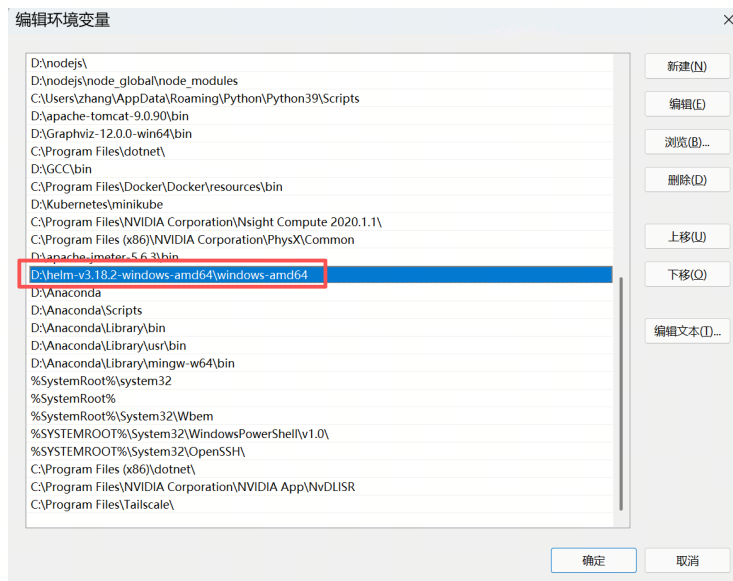
16.49MB



若以上两个版本不适合自己的电脑，请从这里下载符合自己电脑版本的压缩包：

<https://github.com/helm/helm/releases>

然后解压压缩包（免安装，直接解压即可），将该软件目录下的windows-amd64(根据实际情况来)目录放入Path系统环境变量，如下图：



打开命令行，输入：

代码块

```
1 helm version
```

若能输出版本号，说明helm已安装成功，如下图：（版本号以实际为准）

```
C:\Users\zhang>helm version
version.BuildInfo{Version:"v3.18.2", GitCommit:"04cad4610054e5d546aa5c5d9c1b1d5cf68ec1f8", GitTreeState:
"clean", GoVersion:"go1.24.3"}
```

2. 安装 ChaosMesh

通过上一步安装的helm，来向k8s集群中安装ChaosMesh。

执行以下命令：

代码块

```
1 helm repo add chaos-mesh https://charts.chaos-mesh.org
2 # 将 Chaos Mesh 官方的 Helm 仓库地址，注册到本地 Helm 仓库列表。让 Helm 能识别、下载
  Chaos Mesh 的安装包。
3
4 helm install chaos-mesh chaos-mesh/chaos-mesh --namespace chaos-testing --
  create-namespace
```

- 5 # 从刚添加的仓库中，部署安装 Chaos Mesh 组件到 K8s 集群。自动新建 chaos-testing 命名空间，并将服务统一安装隔离在该命名空间内。

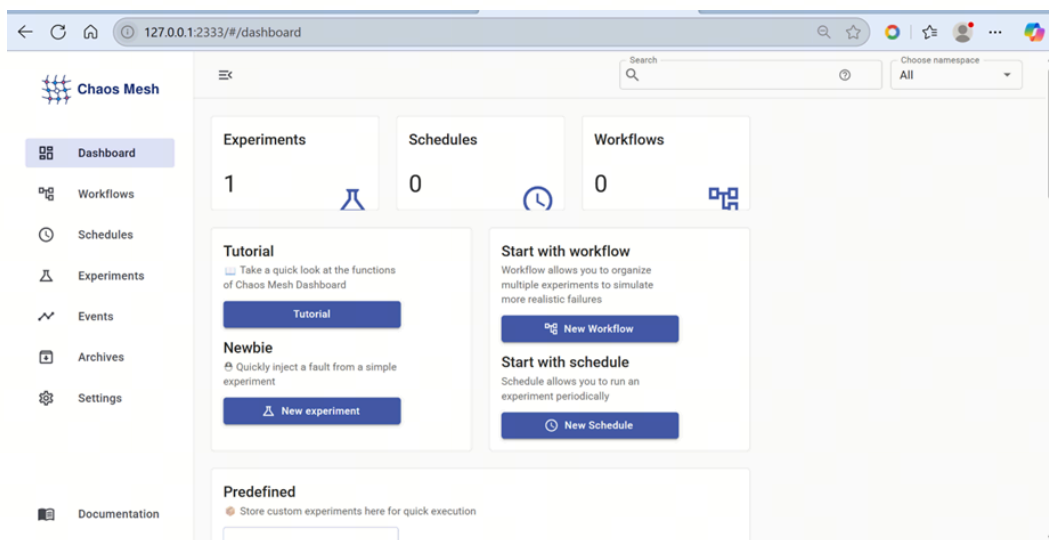
至此，ChaosMesh便安装完成。

执行下面这个命令，以打开ChaosMesh的UI界面：

代码块

- 1 `kubectl port-forward -n chaos-testing svc/chaos-dashboard 2333:2333`
- 2 # 将 K8s 集群内 chaos-dashboard 服务的 2333 端口，映射到本地机器的 2333 端口，可以直接在本地浏览器访问集群内的混沌测试控制台

在浏览器中打开：localhost:2333，其前端界面如下：



如果在打开UI时弹出如下提示：

Enter the token (RBAC Authorization) to continue

Don't know how to get the token? [Click here to generate](#)

Name

Please fill the token name

Token

Please fill the secret token

Submit

请点击"Click here to generate"，勾选“Cluster scoped”，Role选择"Manager",然后按照其提示来生成Token。

Token generator

 Generate different tokens by choosing different namespaces and roles.

☒ Cluster scoped

Namespace

default

Choose namespace

Role

Manager

Choose role

填写好后，应如下图所示：

Enter the token (RBAC Authorization) to continue

Don't know how to get the token? [Click here to generate](#)

Name

account-cluster-manager-qcgej

Please fill the token name

Token

eyJhbGciOiJIUzU1NiIsImtpciI6ImNCMF5SFSWmVtMGNRyOt04X0dFamQxWzZGczDRMnTUVUWnRnWFRTlUUY0gyUUUifQ.eyJhdWQiOiIsiaHR0cHM6Yy9rdWJlcmlldGZvZmRlZmF1bHQue3ZjZmNsdXN0ZXlibG9yYVYwWiXSwiZGhlwjlzc3MTE2NDMzLCJpYXQiOiE3NzcxMTI4MzMsmZmlzcy6lmi0dHhZzOjBva3ViZS5uZXRLRlc5kzWZhdWx0LnN2Yy5jbHVzdG9yLm9yY2FslwiIHRpIjozeiRiOWQ1NDEtNGNHNm00Y2dlLTk5MmYtODY1ZGM3NTEyNWwEwliwia3ViZS5uZXRLcy5pbYl6eyJuYV1lc3BhY2UuOiJkZWZhdWx0liwic2YydmjZWFJyY291bnQzOisibmFtZS16ImFjY291bnQtY2x1c3Rlci1tYW5hZ2YyLXZlV2liwidWkjaWJkZTA1NGM1OGUuNTY0ZS00MmU0LWfkM2EtNDYnWkYyWU3YtXg1N19LCJyYmY0IjE3NzcxMTI4MzMslinN1Yi6lnN5C3R1bTzZSj2aWNIWVWJn63VudDpkZWZhdWx0OmFjY291bnQtY2x1c3Rlci1tY291b2V2YyLXJfZ2VqLn0.VPB8K9-byIVS0iyibHbcOKj8MZDtt-E9-

Please fill the secret token

Submit

3. 创建故障注入实验并执行

有两种方式，选其中一种进行实验即可：

方式1:

通过 YAML 文件来定义故障实验。在sockshop项目目录下新建一个yaml文件，在其中定义你想要注入的故障，比如，模拟杀死某个 Pod：

```
1  kind: PodChaos
2  apiVersion: chaos-mesh.org/v1alpha1
3  metadata:
4    namespace: sock-shop
5    name: pod-kill
6  spec:
7    selector:
8      namespaces:
9        - sock-shop
```

```
10 mode: one
11 action: pod-kill
```

通过 kubectl 执行实验：

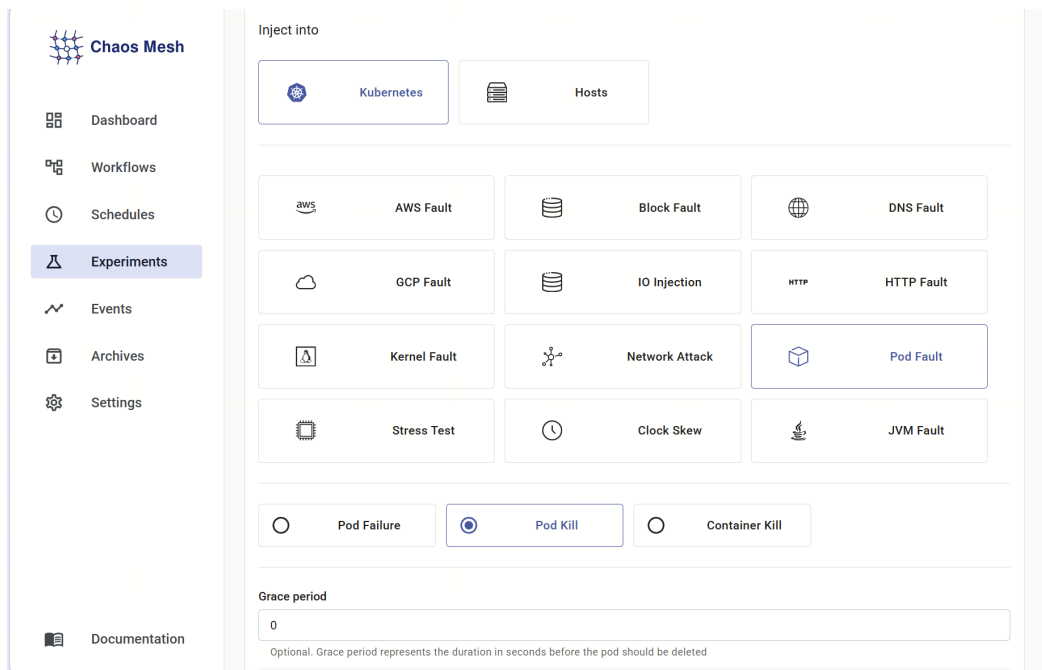
代码块

```
1 kubectl apply -f chaos-experiment.yaml
```

方式2：

通过ChaosMesh的UI来注入故障并执行。

在Experiment部分中可以通过Inject Info来选择想要注入的故障类型，通过Experiment Info来设置该类型下的具体参数。以上是以随机杀死一个pod为例。



Experiment Info

Scope

Namespace Selectors

sock-shop

Specifies the namespaces to which target Pods belong.

Label Selectors

Specifies the labels that target Pods come with.

More Options

Mode

Random One

Specifies the mode of the experiment.

Preview of Pods to be injected

Checking or unchecking Pods to further limit the scope.

	Name	Namespace	IP address	Status
☒	carts-5587489766-xbftz	sock-shop	10.244.0.3	Running

Metadata

Name

pod-kill

Fill the experiment name.

More Options

Run

Run continuously

Duration

Supported formats of the duration are: ms / s / m / h.

Submit

定义好后，如下图，点击Submit，即可将故障注入到k8s集群中。

Search

New experimentLoad fromBy YAML

Inject into

Experiment Info

Submit

All steps are completed

Submit

4. 查看实验状态

在Experiment部分，课查看刚刚注入故障的实验状态，"Completed"说明故障已经成功注入。

Chaos Mesh

DashboardWorkflowsSchedulesExperimentsEventsArchives

+ New experimentBatch operation

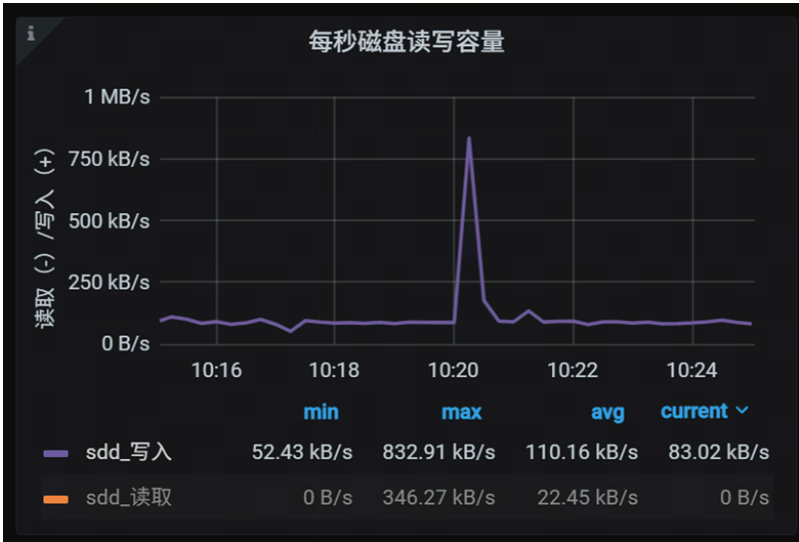
POD FAULT

Completed pod-kill 7f130bad-b48d-4fc2-b792-0f1...Created at 20 seconds ago

5. 分析结果

通过部署的监控服务，观察系统在注入故障前后的表现。

在Grafana中可视化查看相关指标数据，以磁盘每秒读写容量这个指标为例，由于杀死一个Pod，会导致这个Pod的终止与重启操作，进而触发日志相关的读写操作与网络存储卷的元数据更新，最终会导致这个被监控指标的明显变化。



三、实验报告

请根据上面的Prometheus+Grafana部署与ChaosMesh故障注入教程，监控SockShop，在成功部署并监控且注入故障后后，撰写实验报告。以下为实验报告内容组成参考：

1. 实验背景
2. 实验内容
3. 实验步骤（详写）
 - 详细描述Prometheus+Grafana部署+实时监控+数据采集+ChaosMesh故障注入等的步骤和操作
4. 实验结果与分析
 - 展示实验过程中的关键结果，如：
 - **Prometheus抓取的数据：**可以展示Prometheus采集到的部分原始数据或时间序列。
 - **Grafana仪表盘：**展示Grafana中监控微服务的可视化图表。
 - **ChaosMesh故障定义：**展示你定义的故障内容
 - 分析实验结果，讨论监控数据背后的含义，数据如何体现你注入的故障
 - 吞吐量和响应时间：是否符合预期？
 - 资源利用率（如CPU、内存、磁盘I/O）
 - 服务的健康状态：是否有任何异常或故障指标需要关注？等等.....
5. 实验总结

- 总结本次实验的主要收获，反思操作中学到的关键技能。
- 如在实验中遇到问题，可以写下你解决问题个过程